
AI & ML Interview Prep

200-Question Bank with Answers

Generative AI · Machine Learning · Deep Learning

80 GenAI · 60 ML · 60 DL questions across Easy / Medium / Hard

Topics: Pretraining · Fine-Tuning · RAG · Agents · MCP · Model Gateways · Quantization · and more

How to Use This Guide

This question bank is designed for engineers and researchers preparing for roles in applied AI, ML engineering, research science, and MLOps. Questions are grouped by domain and difficulty:

EASY

Foundational concepts. Expected of all candidates.

MEDIUM

Design and depth questions. Show understanding of trade-offs.

HARD

System design, theory, and production challenges. Senior/Staff level.

Answers are intentionally concise—use them as revision anchors and expand with your own examples. For system design questions, practice drawing diagrams and justifying trade-offs aloud.

Section 1: Generative AI

80 questions covering LLM fundamentals, pretraining, fine-tuning (LoRA, RLHF, DPO), RAG pipelines, agents, MCP, model gateways, inference optimisation, hallucination, quantization, speculative decoding, and production serving.

Q1.

EASY

What is a large language model (LLM) and how does it differ from traditional NLP models?

Answer

LLMs are neural networks (typically Transformer-based) trained on massive text corpora to predict the next token. Unlike traditional NLP models that are task-specific and rule-based or trained on small corpora, LLMs learn general language representations and can perform many tasks zero-shot through prompting.

Q2.

EASY

What is tokenization and why does it matter?

Answer

Tokenization converts raw text into discrete tokens (subwords, words, or characters) that the model operates on. Token choice affects vocabulary size, sequence length, multilingual coverage, and cost. BPE and SentencePiece are common algorithms that balance vocabulary size vs. sequence length.

Q3.

EASY

What is a prompt and what is prompt engineering?

Answer

A prompt is the input text given to an LLM. Prompt engineering is the craft of designing prompts—via instructions, examples (few-shot), chain-of-thought hints, or persona setting—to reliably elicit desired outputs without changing model weights.

Q4.

EASY

What is temperature in LLM generation and how does it affect output?

Answer

Temperature T scales logits before softmax: low T (< 1) makes the distribution sharper (more deterministic/focused); high T (> 1) flattens it (more diverse/random). $T=0$ is greedy decoding; $T \sim 0.7$ is common for balanced creativity.

Q5.

EASY

What is the difference between zero-shot, one-shot, and few-shot prompting?

Answer

Zero-shot: no examples, task described in words only. One-shot: one example in the prompt. Few-shot: a handful of examples. More examples generally improve performance by illustrating the expected format and reasoning style.

Q6.

EASY

What is hallucination in LLMs?

Answer

Hallucination is when a model generates plausible-sounding but factually incorrect or fabricated content. It arises because LLMs are trained to produce fluent text, not verified facts. Causes include knowledge gaps, distribution shift, and over-confident token prediction.

Q7.

EASY

What is an embedding and what are they used for?

Answer

An embedding is a dense real-valued vector representing text (token, sentence, or document) in a semantic space. Similar meanings map to nearby vectors. Uses: semantic search, clustering, retrieval, classification, and as input to downstream models.

Q8.

EASY

What is RLHF?

Answer

Reinforcement Learning from Human Feedback: (1) supervised fine-tuning on demonstrations, (2) a reward model trained on human preference rankings, (3) RL (PPO) to fine-tune the LLM to maximize the reward model's score. Used by ChatGPT, Claude, Gemini to align outputs with human preferences.

Q9.

EASY

What is the context window of an LLM?

Answer

The maximum number of tokens (input + output) the model can process in one forward pass. Limited by quadratic memory cost of attention. Larger windows allow longer documents but cost more compute and memory.

Q10.

EASY

What is the difference between encoder-only, decoder-only, and encoder-decoder models?

Answer

Encoder-only (BERT): bidirectional attention, good for classification/embedding. Decoder-only (GPT, Claude): causal attention, good for generation. Encoder-decoder (T5, BART): encoder reads input with bidirectional attention; decoder generates output autoregressively—ideal for translation and summarisation.

Q11.

EASY

What is fine-tuning?

Answer

Continuing training of a pretrained model on a smaller, task-specific dataset. Updates all or a subset of weights so the model adapts to new distribution or task. Cheaper than pretraining but requires labeled data and risks catastrophic forgetting.

Q12.

EASY

What is RAG (Retrieval-Augmented Generation)?

Answer

RAG combines a retriever (searches a knowledge base for relevant chunks) with a generator (LLM). Retrieved context is appended to the prompt, grounding the model's response in up-to-date or private information without retraining the LLM weights.

Q13.

EASY

What is an API gateway for LLMs?

Answer

A middleware layer that sits between clients and LLM backends. It handles routing, authentication, rate limiting, cost tracking, logging, caching, and load balancing across multiple models or providers. Examples: LiteLLM, AWS Bedrock gateway, Kong AI gateway.

Q14.

EASY

What does 'autoregressive' generation mean?

Answer

Tokens are generated one at a time, each conditioned on all previous tokens: $P(x_1..x_n) = \prod P(x_i | x_{<i})$. This makes generation sequential—parallelising output tokens is non-trivial (see speculative decoding).

Q15.

EASY

What is top-k and top-p (nucleus) sampling?

Answer

Top-k: at each step sample only from the k highest-probability tokens. Top-p (nucleus): sample from the smallest set of tokens whose cumulative probability $\geq p$. Nucleus sampling adapts the candidate pool dynamically, often producing more coherent text than fixed-k.

Q16.

EASY

What is the attention mechanism in a Transformer?

Answer

Attention computes a weighted sum of values V using similarity scores between queries Q and keys K : $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$. Multi-head attention applies this in parallel across h heads, each learning different relationships.

Q17.

EASY

What is a vector database and why is it used in GenAI?

Answer

A database optimised for storing and querying high-dimensional embedding vectors via approximate nearest-neighbour (ANN) search (e.g., FAISS, Pinecone, Weaviate). Used in RAG to retrieve semantically similar documents quickly at scale.

Q18.

EASY

What is instruction tuning?

Answer

Fine-tuning on (instruction, response) pairs to teach a model to follow natural-language directives. Results in 'instruction-following' models like InstructGPT/ChatGPT that generalise to unseen instructions better than base LLMs.

Q19.

EASY

What is a system prompt?

Answer

A hidden or prepended prompt that sets the model's persona, constraints, and behaviour before the user's message. It shapes tone, safety boundaries, and task focus across a conversation without user visibility.

Q20.

EASY

What is model distillation in the context of LLMs?

Answer

Training a smaller 'student' model to mimic the output distribution (soft logits or intermediate representations) of a larger 'teacher' model. Yields compact models with performance closer to the teacher than training from scratch on hard labels alone.

Q21.

EASY

What is greedy decoding and what is its main drawback?

Answer

Greedy decoding picks the highest-probability token at every step. It is fast and deterministic but often produces repetitive, locally optimal but globally sub-optimal sequences because it never explores alternative token paths.

Q22.

EASY

What is beam search?

Answer

Beam search keeps the top-B partial sequences (beams) at each step, expanding each and retaining the B highest-scoring continuations. It explores more of the output space than greedy decoding but is $B \times$ more expensive and still does not guarantee global optimum.

Q23.

EASY

What are tokens-per-second (TPS) and time-to-first-token (TTFT)?

Answer

TPS measures throughput: how many tokens the model generates per second. TTFT measures latency: how long until the first token appears. TTFT is dominated by the prefill phase; TPS is dominated by decoding. Both matter for user experience.

Q24.

EASY

What is LangChain?

Answer

An open-source framework for building LLM applications. It provides abstractions for chains, agents, memory, tools, and retrievers, making it easier to compose LLM calls with external data sources and APIs.

Q25.

EASY

What is the difference between a base model and a chat model?

Answer

A base model is trained purely on next-token prediction on large text corpora—it completes text. A chat model is further fine-tuned (often with RLHF/instruction tuning) to follow instructions, maintain dialogue, and refuse harmful requests.

Q26.

MEDIUM

How does KV caching improve LLM inference speed?

Answer

During autoregressive decoding the attention keys and values for all previous tokens are recomputed at every step. KV caching stores these tensors after the first forward pass and retrieves them on subsequent steps, reducing per-step compute from $O(n)$ to $O(1)$. Memory grows linearly with sequence length and batch size; optimisations like multi-query attention (MQA) and grouped-query attention (GQA) reduce the memory footprint.

Q27.

MEDIUM

Explain Mixture of Experts (MoE) architecture.

Answer

MoE replaces dense FFN layers with N expert FFN sub-networks and a learned router that selects top- k experts per token. Only k/N of the parameters are activated per token, enabling models with many parameters (e.g., 1T+) to run at the FLOP cost of a much smaller dense model. Challenges: load balancing (auxiliary losses), communication overhead in distributed settings, and expert collapse. Examples: Mixtral 8×7B, GPT-4 (rumoured), Gemini 1.5.

Q28.

MEDIUM

Explain the pretraining pipeline of a large language model.

Answer

Steps: (1) Data collection and cleaning (web crawl, dedup, quality filter). (2) Tokenizer training (BPE/SentencePiece). (3) Model initialisation. (4) Distributed training—data parallelism, tensor parallelism, pipeline parallelism. (5) Learning rate scheduling (warmup + cosine decay). (6) Checkpoint evaluation on held-out sets and benchmarks. Typically runs for trillions of tokens over weeks to months on thousands of GPUs.

Q29.

MEDIUM

What is LoRA and why is it popular for fine-tuning?

Answer

Low-Rank Adaptation: instead of updating full weight matrices W , LoRA adds low-rank trainable matrices $B \cdot A$ (rank $r \ll d$). Only B and A are trained; W stays frozen. Parameter count drops dramatically (e.g., $r=8$ for a 4096-dim layer $\rightarrow$ 65K vs 16M params). Fast to train, easy to swap, and multiple LoRA adapters can be merged or switched at inference time. QLoRA extends this by quantizing the base model to 4-bit.

Q30.**MEDIUM**

What is speculative decoding?

Answer

A draft ('speculator') model generates k tokens quickly. The target large model verifies all k tokens in one parallel forward pass. Accepted tokens are kept; the first rejected token's correction is used and drafting restarts. Speeds up decoding 2–4× without changing output distribution. The draft model can be a smaller version of the target or a separate lightweight model.

Q31.**MEDIUM**

How would you detect hallucinations automatically?

Answer

Approaches: (1) Self-consistency: sample N answers and flag low agreement. (2) Retrieval grounding: check each claim against retrieved evidence with an NLI model. (3) Uncertainty estimation: aggregate token-level entropy or probe model's own confidence. (4) Factual verification models (e.g., FActScorer) that decompose claims and verify each. (5) LLM-as-judge: a second model critiques the output. No single method is perfect; ensembling helps.

Q32.**MEDIUM**

Explain how RAG works end-to-end.

Answer

Offline: documents → chunking → embedding → vector DB. Online: (1) embed user query, (2) ANN search retrieves top- k chunks, (3) chunks injected into LLM prompt as context, (4) LLM generates grounded answer. Variants: HyDE (generate hypothetical doc first), reranking (cross-encoder refines retrieved chunks), iterative RAG, self-RAG (model decides when to retrieve).

Q33.**MEDIUM**

What are the trade-offs between fine-tuning and RAG?

Answer

Fine-tuning: bakes knowledge into weights, faster inference, no retrieval latency, but requires labeled data, expensive to update, and risks forgetting. RAG: knowledge stays external and updatable, no GPU-intensive training, good for long-tail and proprietary data, but adds latency, retrieval errors propagate, and context window limits apply. Hybrid approaches (fine-tune + RAG) are common in production.

Q34.**MEDIUM**

Explain attention complexity and why it becomes expensive.

Answer

Standard self-attention is $O(n^2 \cdot d)$ in time and $O(n^2)$ in memory due to the full $n \times n$ attention matrix. At $n=128k$ tokens this is ~16B entries. Solutions: FlashAttention (fused kernel, $O(n)$ memory via chunking), sparse attention (attend to local window + global tokens), linear attention approximations, and sliding-window attention (Mistral).

Q35.**MEDIUM**

What is model quantization and why does it matter?

Answer

Quantization reduces weight (and optionally activation) precision from FP32/BF16 to INT8, INT4, or lower. Benefits: smaller memory footprint (4-bit halves vs FP16), faster memory bandwidth-bound inference on consumer GPUs. Techniques: post-training quantization (PTQ), quantization-aware training (QAT), GPTQ, AWQ. Trade-off is slight accuracy loss, especially at 4-bit for smaller models.

Q36.**MEDIUM**

What is an AI agent and how does it differ from a plain LLM call?

Answer

An agent uses an LLM as a reasoning engine in a loop: perceive (observation), think (plan/reason), act (call tools/APIs), observe result, repeat until goal achieved. Unlike a single LLM call, agents handle multi-step tasks, maintain state, and use external tools. Frameworks: ReAct, Plan-and-Execute, OpenAI Assistants, LangGraph.

Q37.**MEDIUM**

What is the Model Context Protocol (MCP) and why was it introduced?

Answer

MCP (Anthropic, 2024) is an open protocol for connecting AI assistants to external data sources and tools. It defines a standard interface so any MCP-compatible tool (file system, database, API) can plug into any MCP-compatible host (Claude Desktop, IDEs). Goal: replace bespoke integrations with a universal, secure, composable connector layer—analogueous to USB for AI tools.

Q38.**MEDIUM**

How does a model gateway / LLM router work?

Answer

A gateway intercepts API calls and routes them to appropriate backends based on rules: cheapest model that meets SLA, load balancing, fallback on error, semantic routing (classify intent → pick model). It also handles: unified API surface across providers (OpenAI, Anthropic, Gemini), caching repeated prompts, cost attribution, and logging. Examples: LiteLLM, Portkey, AWS Bedrock.

Q39.**MEDIUM**

What is DPO (Direct Preference Optimization)?

Answer

DPO re-frames RLHF's reward model + RL loop as a single supervised objective. Given preference pairs (chosen y_w , rejected y_l) the model is trained directly with a binary cross-entropy-style loss that implicitly encodes the reward signal. Simpler to implement, more stable than PPO, and competitive in practice.

Q40.

MEDIUM

Explain chain-of-thought (CoT) prompting.

Answer

CoT adds intermediate reasoning steps to demonstrations, e.g., 'Let's think step by step.' The model learns to externalise its reasoning process, improving accuracy on multi-step arithmetic, logical reasoning, and commonsense tasks. Zero-shot CoT uses the phrase itself; few-shot CoT provides worked examples. Works because intermediate tokens give the model extra computation 'budget'.

Q41.

MEDIUM

What is context stuffing and what are its limits?

Answer

Context stuffing means placing all relevant information (docs, history, examples) in the prompt's context window. Advantages: no retrieval, simple. Limits: (1) cost scales with token count, (2) 'lost in the middle' phenomenon—models pay less attention to middle tokens, (3) context length limit, (4) latency of long prefills.

Q42.

MEDIUM

What is parameter-efficient fine-tuning (PEFT)?

Answer

PEFT methods update only a small subset of parameters while freezing the rest. Variants: LoRA (low-rank adapters), prefix tuning (trainable prefix tokens), prompt tuning (soft prompts), IA³ (learned scaling vectors). Enables task-specific adaptation with 0.1–1% of trainable parameters, making multi-task serving via adapter swapping feasible.

Q43.

MEDIUM

What is self-RAG?

Answer

Self-RAG (Asai et al., 2023) trains a model to generate special reflection tokens that determine when to retrieve, whether retrieval is relevant, and whether the output is grounded. The model itself decides retrieval necessity rather than always retrieving, improving efficiency and relevance.

Q44.

MEDIUM

What is the difference between semantic search and keyword search?

Answer

Keyword (lexical) search matches exact or stemmed terms (BM25, TF-IDF). Semantic search embeds queries and documents and retrieves by cosine similarity in vector space, capturing paraphrase and synonym matches. Hybrid search combines both signals via reciprocal rank fusion (RRF) and typically outperforms either alone.

Q45.**MEDIUM**

What is flash attention and why is it important?

Answer

FlashAttention (Dao et al., 2022) is a GPU-kernel-level optimisation that computes exact attention without materialising the full $n \times n$ matrix. It tiles computation to fit in SRAM, reducing HBM reads/writes from $O(n^2)$ to $O(n)$, cutting memory by $\sim 10\text{--}20\times$ and wall-clock time by $2\text{--}4\times$. Critical for long-context models.

Q46.**MEDIUM**

How does multi-head attention differ from single-head attention?

Answer

Multi-head attention projects Q, K, V into h subspaces, runs attention in each, concatenates results, and projects back. Each head can specialise in different types of relationships (syntactic, coreference, positional). Single-head with the same total dimensions would learn only one relational pattern per layer.

Q47.**MEDIUM**

What is retrieval augmented generation vs. long-context LLMs?

Answer

Long-context LLMs (128k–1M tokens) can ingest entire document sets without retrieval but cost $O(n^2)$ attention compute and show 'lost in the middle' degradation. RAG has lower context cost but introduces retrieval errors. Hybrid: embed documents, retrieve top chunks, fit within a moderate context window.

Q48.**MEDIUM**

Explain tensor parallelism and pipeline parallelism for LLM training.

Answer

Tensor parallelism splits individual weight matrices across devices (e.g., split attention heads across GPUs), requiring all-reduce communication within each layer. Pipeline parallelism partitions layers across devices, sending activations between stages (pipeline bubbles are the inefficiency). Both enable training models larger than single-device memory. Megatron-LM combines both.

Q49.**MEDIUM**

What is PEFT adapter merging and why is it useful?

Answer

Multiple LoRA adapters (each trained for a different task/domain) can be merged into the base model weights at inference time: $W' = W + \alpha \cdot B \cdot A$. This allows serving a single model that incorporates task-specific knowledge without separate adapter loading overhead. Task Arithmetic and TIES merging extend this to combine conflicting adapters gracefully.

Q50.

MEDIUM

What is the difference between InstructGPT and ChatGPT from a training perspective?

Answer

Both use supervised fine-tuning on human demonstrations + PPO with a human-preference reward model (RLHF). ChatGPT extends InstructGPT with a dialogue-specific format and additional safety fine-tuning optimised for multi-turn conversation, making it more robust in chatbot settings.

Q51.

MEDIUM

What is constitutional AI (CAI)?

Answer

Anthropic's approach where the model critiques and revises its own outputs against a set of principles ('constitution') during training. Reduces reliance on human labelers for harmful content. Two phases: supervised learning from self-critique revisions, then RLAI (RL from AI feedback) using the constitution to rank outputs.

Q52.

MEDIUM

How would you build a document Q&A system for a company's internal knowledge base?

Answer

Pipeline: (1) ingest docs (PDF, DOCX, wiki), (2) chunk with overlap, (3) embed with a retrieval-tuned model, (4) store in vector DB with metadata, (5) user query → embed → retrieve top-k → rerank, (6) inject chunks into LLM prompt with citation instructions, (7) return answer + sources. Add access control per chunk for sensitive docs.

Q53.

MEDIUM

What is function calling / tool use in LLMs?

Answer

The model is given a schema of available functions (name, description, parameters). When appropriate it outputs a structured JSON call instead of plain text. The host executes the function, returns results, and the model uses them to produce a final answer. Enables reliable structured output and real-world action (APIs, databases, code execution).

Q54.

MEDIUM

What is prompt injection and how can it be mitigated?

Answer

Prompt injection is an attack where malicious content in retrieved data or user input overrides system instructions (e.g., 'Ignore previous instructions and ...'). Mitigations: input sanitization, privilege separation (never mix untrusted data with instruction-level context), output filtering, and fine-tuning models to be robust to injection patterns.

Q55.**MEDIUM**

What is GPTQ and how does it enable 4-bit inference?

Answer

GPTQ (Frantar et al., 2022) is a one-shot PTQ method that minimises layer-wise reconstruction error. It applies the Hessian-based Optimal Brain Quantisation framework: quantize weights one by one, updating remaining weights to compensate. Achieves near-FP16 perplexity at INT4, enabling 70B models to run on a single 48GB GPU.

Q56.**HARD**

How would you serve an LLM to millions of concurrent users? Describe the full stack.

Answer

Stack: (1) Model optimisation: quantize (INT8/4), compile (TensorRT-LLM, vLLM), enable KV cache, use continuous batching. (2) Serving layer: vLLM or TGI with PagedAttention, autoscaling on GPU clusters (K8s + KEDA). (3) Gateway: load balancer, rate limiter, semantic cache (GPTCache), prompt router. (4) Observability: token cost, latency p50/p99, error rates. (5) Fault tolerance: multi-region deployment, fallback models. PagedAttention is key: it manages KV cache like OS virtual memory, maximising GPU utilisation with continuous batching.

Q57.**HARD**

Explain PagedAttention and why it matters for LLM serving.

Answer

KV cache memory is wasted with static allocation (reserved worst-case length per sequence). PagedAttention (vLLM) borrows OS paging: KV cache divided into fixed-size blocks mapped via a block table. Physical blocks allocated on demand, freed when a sequence ends. Enables: near-zero internal fragmentation, memory sharing for parallel sampling/beam search, and 2–4× throughput improvement. Continuous batching then packs many sequences into one GPU pass, further increasing utilisation.

Q58.**HARD**

Explain the full RLHF training pipeline with PPO in detail.

Answer

Phases: (1) SFT: fine-tune base LLM on curated demonstrations to get π_{SFT} . (2) Reward modelling: collect comparison data ($y_w > y_l$), train R_ϕ with Bradley-Terry loss. (3) PPO: use R_ϕ as reward; optimise π_θ via policy gradient with KL penalty $\lambda \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{SFT}})$ to prevent over-optimisation. PPO clips the probability ratio for stable updates. Challenges: reward hacking, mode collapse, KL-divergence calibration. RLAIIF replaces human raters with a model critic.

Q59.

HARD

What is the 'lost in the middle' problem and how do you engineer around it?

Answer

Models attend more to tokens near the beginning and end of long contexts, ignoring middle content (Liu et al., 2023). Mitigations: (1) Place the most critical context first/last. (2) Use reranking to limit injected chunks to the most relevant. (3) Apply recency-weighted retrieval. (4) Use models fine-tuned for long context (e.g., with 'needle-in-haystack' training). (5) Summarise less relevant context into a shorter form before injection.

Q60.

HARD

How do you handle catastrophic forgetting in continual fine-tuning of LLMs?

Answer

Catastrophic forgetting occurs when fine-tuning overwrites pretrained knowledge. Mitigations: (1) LoRA/PEFT—freeze base weights. (2) Elastic Weight Consolidation (EWC)—penalise changes to weights important for previous tasks (Fisher information). (3) Replay—mix fine-tuning data with samples from pretraining distribution. (4) Modular approaches—task-specific adapter modules. (5) Learning rate: very small LR with few epochs.

Q61.

HARD

Explain how multi-agent systems work and what failure modes exist.

Answer

Multi-agent systems have specialised agents (planner, retriever, coder, critic) that communicate via messages. Orchestrator delegates subtasks; agents return results used by subsequent agents. Failure modes: (1) Error propagation—one agent's mistake cascades. (2) Infinite loops—agents ping-pong without progress. (3) Conflicting instructions between agents. (4) Context fragmentation across agents. (5) Tool call failures not handled gracefully. Mitigations: structured communication protocols, timeout/retry logic, human-in-the-loop checkpoints, and reflection agents.

Q62.

HARD

What is Reinforcement Learning from AI Feedback (RLAIF) and how does it scale?

Answer

RLAIF replaces expensive human preference labelling with a strong AI model (e.g., Claude) as the labeller. The AI judge rates responses per a constitution/rubric, generating preference pairs for reward model training. Scales cheaply vs. human annotation. Risk: if the judge model has biases, they amplify into the trained model. Constitutional AI (Anthropic) combines RLAIF with self-critique to reduce this risk.

Q63.**HARD**

How does speculative decoding maintain the same output distribution as the target model?

Answer

The draft proposes k tokens. The target evaluates them in a single forward pass, producing target probabilities $p(x|x<i)$. Acceptance criterion: token accepted if $u \sim \text{Uniform}(0,1) < \min(1, p_{\text{target}} / p_{\text{draft}})$. If rejected, a corrected token is sampled from $(p_{\text{target}} - p_{\text{draft}})$ normalised—this is the modified rejection sampling step. The resulting distribution is provably identical to the target's autoregressive distribution, guaranteeing no quality loss.

Q64.**HARD**

What are the latency and throughput trade-offs in LLM serving, and how do you optimise both?

Answer

Latency = TTFT + (decode steps $\times$ per-step time). TTFT dominated by prefill (parallelisable on GPU). Per-step time is memory-bandwidth-bound. Throughput = tokens/sec across concurrent requests. Optimise: (1) Continuous batching increases GPU utilisation without sacrificing latency SLAs. (2) KV cache quantisation (INT8 KV) shrinks cache, fitting more sequences. (3) Speculative decoding cuts per-step count. (4) Disaggregate prefill and decode across separate GPU pools to avoid head-of-line blocking.

Q65.**HARD**

Explain GQA (Grouped-Query Attention) and MQA (Multi-Query Attention) and their trade-offs.

Answer

Standard MHA: h heads, each with unique Q, K, V . Memory of KV cache = $2 \times h \times d \times n$. MQA (Shazeer, 2019): all Q heads share a single K, V —reduces KV cache $h\times$. Quality drops slightly. GQA (Ainslie et al., 2023): groups of g heads share K, V —interpolates between MHA and MQA. Llama 2/3, Mistral use GQA: $h=32, g=8$ gives $4\times$ KV reduction with near-MHA quality. Critical for serving long contexts and large batches.

Q66.**HARD**

How do you build an evaluation framework for an LLM product?

Answer

Layers: (1) Automated unit evals—task-specific benchmarks (MMLU, HumanEval, MT-Bench), exact-match or regex. (2) Model-based evals—LLM-as-judge scores factuality, helpfulness, safety. (3) Human evals—gold standard, expensive, for regressions. (4) Canary/shadow testing—route 1% traffic to new model, compare logged outputs. (5) Adversarial red-teaming. (6) Slice analysis by topic/language/length. Metrics: accuracy, BLEU/ROUGE (NLG), latency, refusal rate, user ratings. CI/CD integration to gate deployments.

Q67.**HARD**

What is the difference between dense retrieval and sparse retrieval, and when should you use each?

Answer

Sparse (BM25, TF-IDF): term-frequency-based, exact lexical match, no GPU required, good for specific technical terms/IDs, fast. Dense (DPR, E5, BGE): neural embeddings capturing semantic similarity, handles paraphrase, requires GPU, better for natural language queries. Hybrid (BM25 + dense, fused with RRF) beats either alone in most production benchmarks. Use sparse when exact term match is critical (product codes, legal citations); dense when semantic generalisation matters.

Q68.**HARD**

Explain the mechanics of Flash Attention 2 vs Flash Attention 1.

Answer

FA1 tiles the $Q \times K$ multiplication to avoid materialising $n \times n$ attention in HBM, reading/writing $O(n)$ HBM. FA2 further: (1) reduces non-matmul FLOPs by reordering the outer/inner loop, (2) partitions work more evenly across warps eliminating synchronisation overhead, (3) provides better support for variable-length sequences. FA2 achieves $\sim 2\times$ speedup over FA1 and reaches 50–73% of theoretical A100 FLOPs utilisation.

Q69.**HARD**

How would you implement a production RAG system with low hallucination?

Answer

Design: (1) High-quality chunking (semantic chunking > fixed-length). (2) Hybrid retrieval (dense + BM25) + cross-encoder reranking. (3) Citation-enforced prompting: 'Only use the context provided; cite [1],[2]...' (4) Faithfulness check: NLI model verifies each sentence in answer against retrieved chunks; reject/retry if entailment score low. (5) Confidence calibration: display low-confidence answers with warning. (6) Fallback: if retrieval confidence below threshold, say 'I don't know.' Monitor hallucination rate via automated faithfulness metrics post-deployment.

Q70.**HARD**

What is the scaling law and how does it guide LLM pretraining decisions?

Answer

Chinchilla scaling laws (Hoffmann et al., 2022): given a compute budget C , optimal model size $N^* \propto \sqrt{C}$ and optimal training tokens $D^* \propto \sqrt{C}$, with $N^* \approx D^* / 20$. Earlier GPT-3-era models were undertrained (too large for compute). Chinchilla-optimal: 70B model trained on 1.4T tokens. Implication: for fixed compute, prefer smaller models on more data. Inference-efficient models may deviate—train smaller models on more tokens for faster serving (Llama family philosophy).

Q71.**HARD**

Explain PEFT adapter merging via Task Arithmetic.

Answer

Task Arithmetic (Ilharco et al., 2023): compute 'task vector' $\tau_t = \theta_{ft} - \theta_{base}$ for each fine-tuned model. To combine tasks: $\theta_{merged} = \theta_{base} + \lambda \cdot \sum \tau_t$. Vectors can be added (multi-task), subtracted (unlearning), or negated. TIES merging extends this by resolving sign conflicts among task vectors before summing—improving merged model quality. Enables zero-cost multi-task deployment without joint training.

Q72.**HARD**

What is chain-of-thought with self-consistency and how does it improve reliability?

Answer

Self-consistency (Wang et al., 2022): sample N diverse reasoning chains (via temperature > 0), derive the final answer from each chain, and take the majority-vote answer. Improves over single CoT because: errors in individual chains are unlikely to be correlated; correct reasoning paths converge. Effective but N× inference cost. Can be approximated cheaply by sampling 5–10 chains.

Q73.**HARD**

How would you implement real-time streaming for an LLM API?

Answer

Server-side: use server-sent events (SSE) or WebSocket; flush each token immediately post-generation. FastAPI / uvicorn StreamingResponse or gRPC streaming. vLLM supports async streaming natively. Client-side: buffer partial tokens for display, handle reconnect logic. For structured outputs, buffer until delimiter detected before rendering. Latency tuning: disable output buffering, set TCP_NODELAY, minimize serialisation overhead per token.

Q74.**HARD**

What are AI safety risks specific to agentic LLM systems?

Answer

Key risks: (1) Prompt injection via environment (malicious webpage, email). (2) Privilege escalation—agent with file-system access manipulated to exfiltrate data. (3) Unbounded action loops causing resource exhaustion or irreversible side-effects. (4) Misaligned sub-goals in multi-agent setups. (5) Tool call hallucination (calling non-existent or wrong APIs). Mitigations: minimal privilege principle, sandboxed tool execution, human approval gates for irreversible actions, output parsers that validate tool calls, structured monitoring of agent trajectories.

Q75.**HARD**

What is KV cache eviction and how does it affect long-context inference?

Answer

In infinite or very long contexts, KV cache exceeds GPU memory. Eviction strategies: (1) Sliding window—evict oldest tokens (Mistral). (2) Streaming LLM (Xiao et al., 2023)—keep initial 'attention sink' tokens + recent window; shown to maintain perplexity better than pure sliding window. (3) H2O—evict keys/values with lowest accumulated attention scores. Trade-off: evicted tokens cannot be attended to; performance degrades gracefully depending on task and strategy.

Q76.**HARD**

How does LoRA work mathematically and what is the role of rank r ?

Answer

For weight $W \in \mathbb{R}^{d \times k}$, LoRA adds $\Delta W = B \cdot A$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$. B initialised to zero; A to random Gaussian, so $\Delta W = 0$ at start (stable initialisation). Training updates only B , A . Effective rank of $\Delta W \leq r$. Higher r : more expressive but more parameters. $r=4-16$ typically captures enough task-relevant variation. The intrinsic dimensionality hypothesis (Aghajanyan et al.) motivates this: fine-tuning updates lie in a low-dimensional subspace.

Q77.**HARD**

Explain the end-to-end architecture of a production AI coding assistant (e.g., GitHub Copilot-style).

Answer

Components: (1) IDE plugin captures cursor context (file, surrounding code, imports, recent edits). (2) Context assembly: select relevant snippets via retrieval from repo index (FIM—Fill In the Middle format). (3) Request to LLM gateway with `<prefix><suffix><middle>` tokens. (4) Model (code-trained, often 7–34B) generates completion in streaming. (5) Post-processing: filter unsafe code, syntax validation, deduplication against training data. (6) Telemetry: acceptance rate, latency. (7) Periodically fine-tune on user-accepted completions with privacy guardrails.

Q78.**HARD**

What is the 'alignment tax' and how can it be minimised?

Answer

Alignment tax: performance degradation on capability benchmarks caused by safety fine-tuning (RLHF/DPO). The model becomes more cautious, possibly refusing benign requests or giving less precise technical answers. Minimised by: (1) high-quality, diverse preference data with nuanced labels, (2) targeted fine-tuning only on safety-relevant distributions, (3) iterative red-teaming to fix specific over-refusals, (4) Constitutional AI with precision-targeted revisions, (5) DPO with well-calibrated reference models.

Q79.

HARD

How does multi-modal pretraining work for vision-language models?

Answer

Architecture: vision encoder (ViT or CNN) encodes image into patch embeddings; a projection layer maps to LLM token space; LLM then processes interleaved image and text tokens. Pretraining: (1) contrastive alignment (CLIP-style) to align vision/text embedding spaces, (2) generative pretraining on image-caption pairs (next-token prediction on text given image tokens). Fine-tuning on instruction-following VQA data. Examples: LLaVA (linear projection), Flamingo (cross-attention), GPT-4V. Key challenge: efficient representation of high-res images without overwhelming context.

Q80.

HARD

What is retrieval-augmented fine-tuning (RAFT)?

Answer

RAFT (Zhang et al., 2024) fine-tunes a model on (question, retrieved docs, answer) triples where some training examples include distractor documents. The model learns to identify and cite relevant evidence while ignoring noise—combining benefits of RAG (external knowledge) with fine-tuning (format and skill acquisition). Outperforms pure RAG and pure fine-tuning on domain-specific Q&A benchmarks.

Section 2: Machine Learning

60 questions covering supervised/unsupervised learning, regularisation, ensemble methods, optimisation, evaluation metrics, feature engineering, calibration, causal inference, Bayesian methods, and production ML debugging.

Q1.

EASY

What is the difference between supervised, unsupervised, and reinforcement learning?

Answer

Supervised: learn mapping from labelled (x, y) pairs. Unsupervised: discover structure in unlabelled data (clustering, dimensionality reduction). Reinforcement: an agent learns policy by interacting with environment and maximising cumulative reward via trial-and-error.

Q2.

EASY

What is overfitting and how can it be detected?

Answer

Overfitting: model fits training data too closely, capturing noise—high train accuracy, low val/test accuracy. Detection: train/val loss divergence, large gap between train and test metrics. Fixes: more data, regularisation (L1/L2, dropout), early stopping, simpler model.

Q3.

EASY

What is cross-validation and why is it used?

Answer

k-fold CV: split data into k folds; train on k-1, validate on 1, rotate k times, average metrics. Gives more reliable performance estimate than single train/val split, especially for small datasets. Stratified CV preserves class balance per fold.

Q4.

EASY

What is precision vs recall and when do you prefer each?

Answer

Precision = $TP / (TP + FP)$: how many predicted positives are actually positive. Recall = $TP / (TP + FN)$: how many actual positives are caught. Prefer precision when false positives are costly (spam detection); prefer recall when false negatives are costly (cancer screening). F1 = harmonic mean balances both.

Q5.

EASY

What is gradient descent?

Answer

Iterative optimisation: at each step update weights in the direction of the negative gradient of the loss: $\theta \leftarrow \theta - \alpha \cdot \nabla L$. Variants: SGD (one sample), mini-batch (B samples), full-batch. Learning rate α controls step size.

Q6.

EASY

What is a confusion matrix?

Answer

A matrix of true vs. predicted class counts: rows = actual, cols = predicted. Diagonal = correct predictions. Off-diagonal = errors. From it: accuracy, precision, recall, F1, specificity computed per class.

Q7.

EASY

What is the difference between bagging and boosting?

Answer

Bagging (e.g., Random Forest): train B models on bootstrap samples in parallel; average predictions—reduces variance. Boosting (e.g., XGBoost, AdaBoost): train models sequentially, each focusing on previous errors—reduces bias. Bagging is more robust to outliers; boosting often has higher accuracy.

Q8.

EASY

What is regularisation?

Answer

Adding a penalty term to the loss function to discourage overly complex models. L2 (Ridge): penalty = $\lambda||w||^2$ —shrinks weights towards zero, keeps all features. L1 (Lasso): penalty = $\lambda||w||_1$ —produces sparse solutions, performs implicit feature selection.

Q9.

EASY

What is a learning curve and what does it tell you?

Answer

A plot of train/val loss (or accuracy) vs. training size or epochs. High bias: both curves plateau at poor performance. High variance: large train-val gap. Used to diagnose whether more data, a bigger model, or regularisation would help.

Q10.

EASY

What is feature engineering?

Answer

Transforming raw data into informative features: normalisation, binning, encoding categoricals (one-hot, target encoding), creating interaction terms, log transforms for skewed distributions. Critical for classical ML; less so for deep learning which learns features automatically.

Q11.

EASY

What is a decision tree and how does it split?

Answer

A tree that recursively partitions the feature space. Each split chosen to maximise information gain (classification: entropy/Gini impurity; regression: MSE reduction). Leaf nodes predict the majority class or mean value. Prone to overfitting—depth control, pruning, or ensemble methods needed.

Q12.

EASY

What is normalisation vs. standardisation?

Answer

Normalisation (min-max): scales to $[0,1]$. Standardisation (z-score): transforms to zero mean, unit variance. Standardisation preferred when distribution is approximately Gaussian or for algorithms sensitive to magnitude (SVM, linear regression). Normalisation for neural networks with bounded activations (sigmoid).

Q13.

EASY

What is the difference between a generative and discriminative model?

Answer

Generative: models joint $P(x, y)$, can sample new x ; examples: Naive Bayes, GMM, VAE. Discriminative: models conditional $P(y|x)$ directly; examples: logistic regression, SVM, most neural nets. Discriminative typically has higher accuracy on classification; generative enables data synthesis.

Q14.

EASY

What is PCA?

Answer

Principal Component Analysis: linear dimensionality reduction that finds orthogonal axes (principal components) of maximum variance. Computed via SVD of the data matrix. Reduces dimensions while preserving most variance; used for visualisation, noise reduction, and preprocessing.

Q15.

EASY

What is a hyperparameter vs. a model parameter?

Answer

Model parameters (weights, biases) are learned from data during training. Hyperparameters (learning rate, depth, regularisation strength) are set before training and control the learning process. Optimised via grid search, random search, or Bayesian optimisation on a validation set.

Q16.

MEDIUM

Why does Batch Normalization improve training stability?

Answer

BatchNorm normalises activations per mini-batch to zero mean / unit variance, then applies learned scale γ and shift β . Benefits: (1) reduces internal covariate shift—each layer sees a more stable input distribution, allowing higher LR. (2) provides regularisation (mini-batch noise). (3) smooths the loss landscape. At inference it uses running mean/variance estimated during training.

Q17.

MEDIUM

Explain vanishing and exploding gradients.

Answer

During backprop, gradients are products of weight matrices and activation derivatives across layers. Vanishing: if these products are <1 repeatedly, gradients shrink exponentially—early layers learn nothing. Exploding: products $>1 \rightarrow$ gradients blow up $\rightarrow$ NaN. Solutions: careful initialisation (Xavier/He), gradient clipping, batch norm, residual connections, LSTM/GRU gating, or attention instead of RNN.

Q18.

MEDIUM

Explain ROC-AUC and its limitations on imbalanced datasets.

Answer

ROC plots TPR vs. FPR at all thresholds; AUC = area under it (1 = perfect, 0.5 = random). On heavily imbalanced data, low FPR is easy to achieve (many true negatives), inflating AUC. Precision-Recall AUC (PR-AUC) or F1 at optimal threshold better reflect performance on the minority class.

Q19.

MEDIUM

What is covariate shift and how do you handle it?

Answer

Train and test feature distributions differ while $P(y|x)$ stays the same. Causes: data collection bias, temporal drift. Detection: train a classifier to discriminate train vs. test; high accuracy = shift exists. Mitigations: importance weighting (re-weight training samples by $P_{\text{test}}(x)/P_{\text{train}}(x)$), domain adaptation, or collecting representative data.

Q20.

MEDIUM

Explain the bias-variance decomposition.

Answer

Expected prediction error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$. Bias: error from wrong assumptions (underfitting). Variance: error from sensitivity to training set fluctuations (overfitting). As model complexity grows, bias decreases and variance increases. Goal: find the sweet spot. Ensemble methods (bagging) reduce variance; boosting reduces bias.

Q21.**MEDIUM**

What is the curse of dimensionality?

Answer

As feature dimensionality grows, data becomes increasingly sparse—volume grows exponentially. Effects: distance metrics become meaningless (all points equidistant), nearest-neighbour search degrades, models overfit. Mitigations: PCA/dimensionality reduction, feature selection, regularisation, or deep learning that learns compact representations.

Q22.**MEDIUM**

How would you debug model drift in production?

Answer

Steps: (1) Monitor input feature distributions (PSI, KS test) vs. training baseline. (2) Monitor output prediction distributions and confidence scores. (3) Monitor business metrics (conversion, error rates). (4) If ground truth available, track model performance metrics. (5) Root cause: data source change, seasonality, concept drift? (6) Remediate: retrain on recent data, update feature pipeline, or rule-based correction.

Q23.**MEDIUM**

Explain why ensemble methods improve performance.

Answer

Individual models have errors; if errors are uncorrelated, averaging cancels them out. Formally: bias of ensemble $\approx$ bias of individuals; variance of ensemble $= \sigma^2/N$ if perfectly uncorrelated (N members). Random Forest introduces decorrelation via random feature subsets. Boosting reduces bias sequentially. Stacking combines diverse model types via a meta-learner.

Q24.**MEDIUM**

Why does PCA maximise variance?

Answer

PCA finds the direction w that maximises the variance of the projected data: $\max w^T \cdot \text{Cov} \cdot w$ s.t. $\|w\|=1$. The solution is the eigenvector of the covariance matrix with the largest eigenvalue. Subsequent components are orthogonal eigenvectors in decreasing eigenvalue order. Maximising variance preserves the most discriminative spread in the data.

Q25.**MEDIUM**

What is gradient boosting and how does XGBoost improve on it?

Answer

Gradient boosting: fit a weak learner (usually a tree) to the negative gradient (pseudo-residuals) of the loss, add to ensemble with step size η . XGBoost improvements: (1) second-order Taylor expansion of loss (uses Hessian). (2) Regularisation terms on tree structure (L1/L2 on leaf weights). (3) Column/row subsampling. (4) Cache-aware computation for speed. (5) Sparse-aware split finding. Together make XGBoost much faster and more accurate.

Q26.**MEDIUM**

What is the difference between L1 and L2 regularisation mathematically?

Answer

Loss_L2 = $L + \lambda \cdot \sum w^2$. Gradient: $\partial/\partial w = -\partial L/\partial w - 2\lambda w \rightarrow$ weight decay: $w \leftarrow w(1-2\lambda\alpha)$. Shrinks weights proportionally—never exactly zero. Loss_L1 = $L + \lambda \cdot \sum |w|$. Gradient: $\pm \lambda$ (sign of w)—constant shrinkage regardless of magnitude, pushes small weights exactly to zero. L1 induces sparsity; L2 distributes weight.

Q27.**MEDIUM**

What is the EM algorithm?

Answer

Expectation-Maximisation: iterative method for models with latent variables (e.g., GMM). E-step: compute expected value of latent variables given current parameters (soft assignments). M-step: update parameters to maximise expected complete-data log-likelihood. Guaranteed to increase likelihood monotonically; converges to local optimum. Used in k-means (hard EM), HMMs, mixture models.

Q28.**MEDIUM**

What is SHAP and how does it explain model predictions?

Answer

SHAP (SHapley Additive exPlanations) assigns each feature a Shapley value: its average marginal contribution across all possible feature coalitions. Unique solution satisfying efficiency, symmetry, dummy, and additivity axioms. TreeSHAP computes exact values for tree models in $O(TLD^2)$ time. Provides both local (per-prediction) and global (feature importance) explanations.

Q29.**MEDIUM**

What is the difference between MAP and MLE estimation?

Answer

MLE: maximise $P(\text{data}|\theta)$ —finds θ that makes observed data most probable, no prior assumption. MAP: maximise $P(\theta|\text{data}) = P(\text{data}|\theta) \cdot P(\theta)$ —incorporates prior $P(\theta)$. L2 regularisation corresponds to Gaussian prior (MAP); L1 to Laplacian prior. As $\text{data} \rightarrow \infty$, $\text{MAP} \rightarrow \text{MLE}$ (prior washes out).

Q30.**MEDIUM**

What are information gain and Gini impurity?

Answer

Information gain: reduction in entropy after split: $IG = H(\text{parent}) - \sum (|\text{child}|/|\text{parent}|) \cdot H(\text{child})$. Gini impurity: $G = 1 - \sum p^2$. Both measure purity of splits in decision trees. Gini is faster to compute (no log); entropy is slightly more informative. In practice, performance is very similar.

Q31.**MEDIUM**

How does SMOTE address class imbalance?

Answer

SMOTE (Synthetic Minority Oversampling TEchnique): for each minority sample, find its k nearest neighbours in feature space, create synthetic points by interpolating: $x_{\text{new}} = x + \lambda \cdot (x_{\text{neighbour}} - x)$. Increases minority class without simple duplication. Variants: ADASYN (adaptively generates more near boundary), SMOTE-ENN (cleans noisy majority samples after synthesis).

Q32.**MEDIUM**

What is a Gaussian Mixture Model (GMM)?

Answer

GMM: data modelled as mixture of K Gaussians: $P(x) = \sum \pi_k \cdot N(x; \mu_k, \Sigma_k)$. Parameters $\{\pi, \mu, \Sigma\}$ estimated via EM. Unlike k -means (hard assignment), GMM gives soft probabilistic membership. Can model non-spherical clusters (with full Σ). BIC/AIC used to select K .

Q33.**MEDIUM**

What is the difference between parametric and non-parametric models?

Answer

Parametric: fixed number of parameters regardless of data size (linear regression, logistic regression, MLP). Faster, requires distributional assumptions. Non-parametric: number of parameters grows with data (k -NN, kernel SVM, GP, decision trees). More flexible, requires more data/memory, no strong distributional assumptions.

Q34.**MEDIUM**

What is the ROC curve and how is it constructed?

Answer

At each possible classification threshold t , compute $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$ and $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$. Plot (FPR, TPR) for all t . Diagonal = random classifier. AUC = probability that a randomly chosen positive is ranked above a randomly chosen negative. Computed efficiently by sorting scores and integrating trapezia.

Q35.**MEDIUM**

What is time-series cross-validation and why is standard k -fold wrong?

Answer

Standard k -fold randomly shuffles—future data leaks into training, inflating metrics. Time-series CV uses forward chaining: train on $1..t$, validate on $t+1..t+k$, expand and repeat. Or walk-forward with a fixed window. Ensures temporal ordering is respected and no future information leaks.

Q36.**MEDIUM**

What is early stopping and how does it prevent overfitting?

Answer

Monitor validation loss during training; stop when it stops improving (or starts increasing) for p patience epochs. Acts as implicit regularisation—limits effective model capacity. The checkpoint at best validation loss is saved. Combined with learning rate scheduling it is often more effective than L2 alone.

Q37.**MEDIUM**

How does a support vector machine (SVM) work?

Answer

SVM finds the maximum-margin hyperplane separating classes: $\max \text{margin} = 2/\|w\|$ subject to $y(w \cdot x + b) \geq 1$. Solved via quadratic programming. Kernel trick: map to high-dimensional feature space implicitly via $K(x, x') = \phi(x) \cdot \phi(x')$ —enables non-linear boundaries. C parameter controls margin-violation trade-off.

Q38.**MEDIUM**

What is the difference between generalization and memorization?

Answer

Generalisation: model learns patterns that hold on unseen data. Memorisation: model stores training examples verbatim—fails on new data. Modern large models can memorise training data (a privacy/copyright concern). Double-descent phenomenon: beyond interpolation threshold, overparameterised models can generalise despite zero training loss.

Q39.**MEDIUM**

What is multi-label vs. multi-class classification?

Answer

Multi-class: exactly one class per sample from K classes (softmax output). Multi-label: each sample can belong to multiple classes simultaneously (sigmoid per class, independent binary outputs). Multi-label requires different loss (binary cross-entropy per label) and different metrics (per-label precision/recall, micro/macro averaging).

Q40.**MEDIUM**

Explain the difference between batch gradient descent, SGD, and Adam.

Answer

BGD: use full dataset per update—stable but slow. SGD: one sample—fast, noisy, escapes local minima. Mini-batch: B samples—balance. Adam: adaptive per-parameter learning rates using first moment (momentum $m_t = \beta m_{t-1} + (1-\beta)g_t$) and second moment ($v_t = \beta v_{t-1} + (1-\beta)g_t^2$); update $\theta \leftarrow \theta - \alpha \cdot m_t / (\sqrt{v_t} + \epsilon)$. Bias-corrected. Converges fast with less LR tuning.

Q41.**HARD**

What is the double-descent phenomenon?

Answer

Classical wisdom: bias-variance trade-off gives a U-shaped test error. Double-descent: as model capacity increases past the interpolation threshold (zero training error), test error descends again. Explains why overparameterised models (neural nets, transformers) generalise despite fitting training data perfectly. Mechanism: inductive bias of gradient descent finds minimum-norm solution among all zero-loss solutions—implicitly regularised.

Q42.**HARD**

What is conformal prediction and why is it distribution-free?

Answer

Conformal prediction produces prediction sets with guaranteed coverage: $P(Y \in \hat{Y}(X)) \geq 1 - \alpha$ for any distribution and any model. Process: compute nonconformity scores on calibration set, take $(1 - \alpha)$ quantile $q_{1-\alpha}$, prediction set = $\{y: \text{score}(x, y) \leq q_{1-\alpha}\}$. Coverage guaranteed by the exchangeability of calibration and test points—no distributional assumption needed. Useful for reliable uncertainty quantification in production.

Q43.**HARD**

Explain causal inference vs. correlation and the do-calculus.

Answer

Correlation is associational: $P(Y|X=x)$. Causal effect requires intervention: $P(Y|\text{do}(X=x))$ —what Y would be if we forced $X=x$. Pearl's do-calculus is a set of three rules for transforming do-expressions to observational quantities using a causal graph (DAG). Confounders (common causes) create spurious correlations; back-door and front-door criteria identify valid adjustment sets for causal identification without experiments.

Q44.**HARD**

What is Gaussian Process regression and when should you use it?

Answer

GP: defines a distribution over functions; any finite set of outputs is jointly Gaussian. Prior: $f \sim \text{GP}(m(x), k(x, x'))$. Posterior given observations: analytic update via matrix inversion, giving a predictive distribution with mean and uncertainty. $O(n^3)$ training, $O(n^2)$ storage—limits to $\sim 10k$ points. Use when: small data, uncertainty estimates critical, smooth function prior reasonable. Kernels (RBF, Matérn) encode assumptions about smoothness.

Q45.**HARD**

What is variational inference and how does it differ from MCMC?

Answer

VI: approximate intractable posterior $P(z|x)$ with a variational family $q(z;\phi)$; minimise $KL(q||p) \rightarrow$ maximise $ELBO = E_q[\log P(x,z)] - E_q[\log q(z)]$. Converts inference to optimisation—fast, scalable (amortised VI / VAE). MCMC: draws exact (asymptotically) samples from $P(z|x)$ via Markov chains (HMC, Gibbs). Slower but unbiased, no approximation error. Use VI for large-scale; MCMC for rigorous Bayesian inference.

Q46.**HARD**

Explain the Johnson-Lindenstrauss lemma and its applications.

Answer

JL lemma: for any set of n points in R^d and $\epsilon > 0$, there exists a linear map to R^k ($k = O(\log n / \epsilon^2)$) that preserves all pairwise distances within factor $(1 \pm \epsilon)$ with high probability. Random Gaussian or sparse projections achieve this. Applications: approximate nearest-neighbour search, dimensionality reduction for large-scale ML, locality-sensitive hashing, and privacy-preserving data sketches.

Q47.**HARD**

What is the No Free Lunch theorem and what does it imply?

Answer

NFL (Wolpert, 1996): averaged over all possible data-generating distributions, no learning algorithm outperforms any other. Implies: any algorithm that works well on some problems must fail on others. Practical implication: algorithm selection requires knowledge of the problem domain—there is no universally best model. Strong inductive bias that matches the true data distribution is the key to good performance.

Q48.**HARD**

What is the relationship between dropout and Bayesian inference?

Answer

Gal & Ghahramani (2016): training a neural network with dropout and sampling multiple forward passes at test time is equivalent to approximate variational inference in a Bayesian neural network with specific weight priors. Each dropout mask corresponds to a different model sample. MC Dropout: average N stochastic forward passes for a predictive distribution and uncertainty estimate—free approximate Bayesian inference without changing the model architecture.

Q49.**HARD**

How does XGBoost handle missing values?

Answer

XGBoost learns a default direction for each split: during training, for each feature the algorithm tries sending missing values left or right and picks whichever gives the best gain. The chosen direction (sparsity-aware split finding) is stored in the tree. At inference, missing values follow the learned direction. This is more principled than mean imputation and often matches or beats hand-imputed data.

Q50.**HARD**

What is online learning and how does it differ from batch learning?

Answer

Online (incremental) learning: model updated with each new example as it arrives—no retraining from scratch. Algorithms: SGD, passive-aggressive, Vowpal Wabbit. Trade-offs: low memory, handles concept drift natively, but sensitive to outliers and learning rate scheduling. Batch learning: train on full dataset periodically—stable but requires scheduled retraining pipelines for drift. For high-velocity data streams (fraud, ads), online learning is preferred.

Q51.**HARD**

Explain natural gradient descent and why it is more efficient than vanilla gradient descent.

Answer

Vanilla GD uses Euclidean metric in parameter space: $\Delta\theta = -\alpha \cdot \nabla L$. Natural gradient uses the Fisher Information Matrix (FIM) as a Riemannian metric: $\Delta\theta = -\alpha \cdot \mathbf{F}^{-1} \cdot \nabla L$. FIM accounts for the curvature of the model's distribution—equal steps in parameter space correspond to equal KL steps in distribution space. Converges faster with fewer updates. K-FAC approximates $\mathbf{F}^{-1}$ efficiently using Kronecker structure.

Q52.**HARD**

What is the VC dimension and how does it relate to generalisation?

Answer

VC dimension = maximum number of points that can be shattered (labelled arbitrarily correctly) by a hypothesis class. Shattering n points requires 2^n labellings. VC-based bound: generalisation error $\leq$ train error + $O(\sqrt{(d \cdot \log(n)/n)})$ where d = VC dim, n = samples. High VC dim $\rightarrow$ rich class $\rightarrow$ needs more data to generalise. Linear classifiers in $\mathbb{R}^p$: VC dim = $p+1$. Neural nets: VC dim exponential in parameters (but modern neural nets generalise better than bound suggests—see double-descent).

Q53.**HARD**

What is an isotonic regression and when is it used?

Answer

Isotonic regression fits a non-decreasing (or non-increasing) step function to data, minimising squared error subject to the monotonicity constraint. Solved in $O(n)$ via the pool adjacent violators (PAV) algorithm. Applications: (1) calibrating classifier probabilities (when model scores are monotone but miscalibrated), (2) non-parametric monotone trend estimation, (3) multi-dimensional isotonic regression for ranking problems.

Q54.**HARD**

What is the difference between frequentist and Bayesian perspectives on ML?

Answer

Frequentist: parameters are fixed but unknown; inference via MLE, confidence intervals (frequentist coverage). Data is random. Bayesian: parameters are random variables with priors; posterior $P(\theta|\text{data}) = P(\text{data}|\theta)P(\theta)/P(\text{data})$. Credible intervals are probability statements about parameters. Bayesian naturally handles uncertainty, regularises via priors, enables model averaging, but requires prior specification and often intractable posteriors ($\rightarrow$ VI/MCMC).

Q55.**HARD**

Explain the kernel trick and give two examples of kernels.

Answer

The kernel trick computes the inner product in an implicit feature space $\phi(x) \cdot \phi(z)$ via a kernel function $K(x,z)$ without explicitly computing ϕ . Enables non-linear decision boundaries in linear algorithms (SVM, PCA, ridge regression). Examples: (1) RBF/Gaussian: $K(x,z) = \exp(-\|x-z\|^2/2\sigma^2)$ —infinite-dimensional feature space, universal approximator. (2) Polynomial: $K(x,z) = (x \cdot z + c)^d$ —all polynomial features up to degree d . Kernel selection encodes inductive bias.

Q56.**HARD**

What is matrix factorization for recommender systems and what are its limitations?

Answer

Matrix factorisation (SVD, ALS, SGD): decompose user-item rating matrix $R \approx U \cdot V^T$ where $U \in \mathbb{R}^{n \times k}$, $V \in \mathbb{R}^{m \times k}$, k = latent factors. Users and items embedded in shared latent space; predicted rating = $u_i \cdot v_j$. Limitations: (1) cold-start for new users/items without ratings, (2) no side features (demographics, content), (3) assumes static preferences, (4) popularity bias. Neural collaborative filtering and graph-based methods address these.

Section 3: Deep Learning

60 questions covering neural network fundamentals, CNNs, RNNs, Transformers, self-supervised learning, diffusion models, training stability, attention variants, NAS, and theoretical foundations.

Q1.

EASY

What is backpropagation?

Answer

Backprop applies the chain rule to compute gradients of the loss w.r.t. every parameter in the network: $\partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial w$. Gradients flow backward from the output layer to the input. Computations are cached from the forward pass (activations) to avoid redundant work. Foundational to training all neural networks.

Q2.

EASY

What is the vanishing gradient problem in RNNs?

Answer

In RNNs, gradients are backpropagated through time (BPTT). For long sequences the gradient is a product of many Jacobians of the recurrent weight matrix—if spectral radius < 1 , gradients vanish; if > 1 , they explode. Result: RNNs fail to learn long-range dependencies. Solutions: LSTM/GRU (gating), gradient clipping, Transformers.

Q3.

EASY

What is a convolutional neural network (CNN) and why is it suited for images?

Answer

CNNs apply learnable filters via convolution: each filter detects local patterns (edges, textures). Key properties: local connectivity (each neuron sees a local receptive field), weight sharing (same filter applied everywhere $\rightarrow$ translation equivariance), pooling (subsampling $\rightarrow$ spatial invariance). Dramatically fewer parameters than fully-connected on image data.

Q4.

EASY

What is an activation function and why is non-linearity important?

Answer

Activation functions introduce non-linearity, enabling networks to approximate any function (universal approximation theorem). Without them, stacked linear layers collapse to a single linear transformation. Common: ReLU ($f = \max(0, x)$ —fast, sparse), GELU (smooth ReLU, used in transformers), sigmoid (output layer for binary), softmax (multi-class output).

Q5.

EASY

What is dropout and how does it prevent overfitting?

Answer

Dropout randomly zeros each neuron activation with probability p during training. Forces the network to learn redundant representations—no neuron can rely on any other. At inference, all neurons active but outputs scaled by $(1-p)$. Acts as ensemble of 2^n sub-networks. Most effective in large fully-connected layers.

Q6.

EASY

What is the difference between max pooling and average pooling?

Answer

Max pooling: takes the maximum activation in a spatial region—emphasises the most salient feature. Average pooling: computes mean—smoother, preserves more information. Max pooling is more common in hidden layers (strong feature detector); global average pooling before the classifier in modern architectures (GAP layer).

Q7.

EASY

What is weight initialisation and why does it matter?

Answer

Poor initialisation → vanishing/exploding gradients from the start. Xavier (Glorot): $\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$ —good for tanh/sigmoid. He: $\text{Var}(w) = 2/\text{fan_in}$ —good for ReLU. Initialise biases to 0, final layer weights near 0. Proper init ensures signal propagates through the network without exponential growth or decay.

Q8.

EASY

What is a recurrent neural network (RNN)?

Answer

RNN processes sequences by maintaining a hidden state $h_t = f(W_{in}x_t + W_{hh}h_{t-1} + b)$. The same weights are applied at every time step. Can model variable-length sequences. Limitation: vanishing gradients for long sequences → superseded by LSTM/GRU and eventually Transformers.

Q9.

EASY

What is the difference between LSTM and GRU?

Answer

LSTM: 3 gates (input, forget, output) + cell state c_t . More expressive, more parameters. GRU: 2 gates (reset, update), merges cell and hidden state—simpler, fewer parameters. $\text{GRU} \approx \text{LSTM}$ in most tasks with less compute. Both address vanishing gradients via additive cell updates that allow gradients to flow without decay.

Q10.

EASY

What is transfer learning?

Answer

Using weights of a model pretrained on a large dataset (e.g., ImageNet, large text corpus) as initialisation for a new task. Fine-tune all or the last few layers on the target task. Lower data requirements for the downstream task; leverages general features learned during pretraining. Foundational to modern NLP (BERT, GPT) and computer vision (ResNet, ViT).

Q11.

EASY

What is the purpose of a softmax function?

Answer

Softmax converts a vector of raw scores (logits) z_i into a probability distribution: $P(i) = \exp(z_i) / \sum_j \exp(z_j)$. Ensures outputs are non-negative and sum to 1. Used as the final layer for multi-class classification. Temperature-scaled softmax: divide logits by T before softmax (controls sharpness, used in knowledge distillation and LLM sampling).

Q12.

EASY

What is batch size and how does it affect training?

Answer

Batch size B = number of samples per gradient update. Large B : stable gradients, better hardware utilisation, but may converge to sharp minima with worse generalisation and requires LR scaling (linear scaling rule). Small B : noisy gradients act as implicit regularisation, explore loss landscape better, but less GPU efficient. Common: 32–512; very large (16k+) in LLM pretraining with gradient accumulation.

Q13.

EASY

What is semantic segmentation?

Answer

Task: assign a class label to every pixel in an image. Architecture: encoder (e.g., ResNet) extracts features at multiple scales; decoder (e.g., U-Net skip connections, DeepLab ASPP) upsamples to original resolution. Metrics: mean Intersection over Union (mIoU), pixel accuracy. Used in autonomous driving, medical imaging.

Q14.

EASY

What are residual connections?

Answer

ResNet's key innovation: skip connection adds the input x to the output of a block $F(x)$: $\text{output} = F(x) + x$. Allows gradients to flow directly through the skip path—alleviates vanishing gradients in very deep networks. Trains networks of 100+ layers. Also stabilises optimisation by enabling the network to learn residuals rather than full mappings.

Q15.

EASY

What is the role of the learning rate scheduler?

Answer

LR scheduler adjusts learning rate during training. Common schemes: step decay (multiply by γ every n epochs), cosine annealing (LR follows cosine curve from max to min), warmup + decay (ramp up LR for first k steps then decay—standard in transformer training), 1-cycle policy. Proper scheduling prevents overshooting and allows fine-grained convergence.

Q16.

MEDIUM

Why are Transformers better than RNNs for LLMs?

Answer

RNNs process tokens sequentially— $O(n)$ serial steps, limited parallelism during training. Transformers process all tokens in parallel via attention— $O(n^2)$ but parallelisable on GPU. Attention directly connects any two positions (long-range dependency), whereas RNNs must route information through all intermediate steps. Better scaling properties: more data + compute $\rightarrow$ better performance.

Q17.

MEDIUM

Explain positional encoding.

Answer

Transformers have no inherent notion of order. Positional encodings (PE) add position information to token embeddings. Sinusoidal (original): $PE(pos, 2i) = \sin(pos/10000^{(2i/d)})$, $PE(pos, 2i+1) = \cos(\dots)$. Learned PE: trainable embedding per position. RoPE (Rotary PE): rotates Q,K vectors—enables relative position encoding and extrapolation to longer sequences (used in LLaMA, Mistral). ALiBi: adds linear bias to attention scores.

Q18.

MEDIUM

What is masked self-attention?

Answer

Causal (decoder) attention applies a causal mask: position i can only attend to positions $\leq i$. The mask sets attention scores for $j > i$ to $-\infty$ before softmax $\rightarrow$ zero weight. Ensures autoregressive generation: each token can only see previous tokens, not future ones. BERT uses bidirectional attention (no mask) during training with a masked-LM objective.

Q19.

MEDIUM

What is the difference between encoder-only and decoder-only models?

Answer

Encoder-only (BERT, RoBERTa): bidirectional attention over entire input. Best for understanding tasks (classification, NER, QA). Decoder-only (GPT, LLaMA, Claude): causal attention; trained on next-token prediction; generation-native. Best for generation tasks. Encoder-decoder (T5, BART): encoder reads input, decoder generates output—best for seq2seq (translation, summarisation).

Q20.**MEDIUM**

What causes unstable training in deep neural networks?

Answer

Causes: (1) poor initialisation → vanishing/exploding gradients at start, (2) very high learning rate → loss diverges, (3) batch size too small → noisy updates, (4) no normalisation → internal covariate shift, (5) dead ReLU neurons (all outputs < 0), (6) NaN from numerical overflow (especially in mixed precision). Fixes: proper init, gradient clipping, norm layers, LR warmup, weight decay.

Q21.**MEDIUM**

What is temperature scaling in softmax?

Answer

Divide logits by temperature T before softmax. $T < 1$: sharper distribution, more confident (greedy-like). $T > 1$: flatter, more uniform (more random). $T = 1$: standard. Applications: (1) LLM sampling—control creativity. (2) Knowledge distillation—high T softens teacher distributions, revealing inter-class similarities. (3) Calibration—find T on val set that minimises ECE (Expected Calibration Error).

Q22.**MEDIUM**

What is gradient clipping?

Answer

Clip gradients to prevent exploding gradient updates. Two variants: (1) clip by value—cap each gradient element to $[-c, c]$. (2) Clip by norm—if $\|g\| > c$, scale $g \rightarrow g \cdot c / \|g\|$. Clip by norm is preferred as it preserves gradient direction. Essential in RNN training and large-scale transformer training (gradient norm monitoring reveals instabilities early).

Q23.**MEDIUM**

Why does attention work so well?

Answer

Attention provides: (1) direct $O(1)$ path between any two positions (vs. $O(n)$ for RNN). (2) Dynamic context-dependent weighting—attention weights adapt to the input rather than using fixed receptive fields. (3) Interpretability—attention weights hint at what the model is focusing on. (4) Permutation equivariance (with positional encodings)—scales well across different sequence structures.

Q24.**MEDIUM**

What is layer normalisation vs batch normalisation?

Answer

BatchNorm normalises across the batch dimension per feature—depends on batch statistics, tricky for small batches, RNNs, and variable-length sequences. LayerNorm normalises across the feature dimension for each sample independently—batch-size independent, suitable for sequence models and transformers. RMSNorm (used in LLaMA) removes the bias term and mean centring for efficiency.

Q25.**MEDIUM**

What are the main components of a Vision Transformer (ViT)?

Answer

ViT: (1) Patch embedding—divide image into N patches, linearly embed each to d dimensions. (2) Class token prepended. (3) Positional embeddings added. (4) L transformer encoder blocks (self-attention + MLP + LayerNorm). (5) MLP head on the [CLS] token for classification. Requires large data or pretraining; patch size controls trade-off between sequence length and spatial resolution.

Q26.**MEDIUM**

What is contrastive learning and how does SimCLR work?

Answer

Contrastive learning trains encoders to embed similar pairs close and dissimilar pairs far. SimCLR: (1) Two augmentations of each image $\rightarrow$ two views. (2) Encode with shared encoder f + projection head g . (3) NT-Xent loss: maximise agreement between views of the same image vs. all other images in the batch. Large batch (4096+) provides many negatives. After training, discard projection head; use encoder embeddings for downstream tasks.

Q27.**MEDIUM**

What is the vanishing/exploding gradient problem in very deep networks (non-RNN)?

Answer

Gradients flow through many layers via chain rule: $\partial L / \partial W = \partial L / \partial a \cdot J$. If J has singular values < 1 consistently: gradient $\rightarrow 0$. If > 1 : gradient $\rightarrow \infty$. Deep networks with sigmoid/tanh suffer due to saturating derivatives. Solutions: ReLU (gradient = 0 or 1), residual connections (skip gradient paths), layer norm, proper init.

Q28.**MEDIUM**

What is an attention mask in transformers and what are the types?

Answer

Attention masks control which positions attend to which. Types: (1) Padding mask: zeros out attention to padding tokens. (2) Causal (look-ahead) mask: lower-triangular matrix preventing attention to future tokens (decoder). (3) Cross-attention mask: in encoder-decoder, queries (decoder) attend to encoder positions. Masks are applied by adding $-\infty$ to masked positions before softmax.

Q29.**MEDIUM**

What is the GELU activation function and why is it used in transformers?

Answer

GELU (Gaussian Error Linear Unit): $x \cdot \Phi(x)$ where Φ is the standard normal CDF. Approximated as $0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$. Unlike ReLU: smooth and non-zero gradient everywhere, slight negative output for small negative x . Empirically outperforms ReLU in language models. Used in BERT, GPT, and most modern transformers.

Q30.**MEDIUM**

What is U-Net and why is it popular for medical imaging?

Answer

U-Net: encoder-decoder architecture with skip connections concatenating encoder feature maps to decoder at matching resolution. Encoder: contracting path (conv + pool). Decoder: expanding path (upsample + conv). Skip connections preserve fine spatial details lost during pooling. Excels with very limited labelled data (common in medical imaging). Trained with binary or multi-class cross-entropy + dice loss.

Q31.**MEDIUM**

What is knowledge distillation in deep learning?

Answer

Train a smaller student network to match a larger teacher's soft output probabilities (at temperature $T > 1$) rather than hard labels. Loss = $\alpha \cdot \text{CE}(y, p_{\text{student}}) + (1-\alpha) \cdot \text{KL}(p_{\text{teacher}} || p_{\text{student}})$. Soft targets encode inter-class similarities (e.g., 'cat' looks somewhat like 'dog'). Student often outperforms training from scratch on hard labels alone. Used to create efficient models: DistilBERT, TinyBERT.

Q32.**MEDIUM**

What is depthwise separable convolution?

Answer

Standard convolution: $C_{\text{in}} \times C_{\text{out}}$ filters of size $K \times K$. Depthwise separable: (1) Depthwise conv: C_{in} filters of $K \times K$, one per channel. (2) Pointwise conv: 1×1 convolution to mix channels. Computation ratio: $1/C_{\text{out}} + 1/K^2$ vs. standard—8-9× cheaper for $K=3$. Used in MobileNet, Xception for efficient on-device inference.

Q33.**MEDIUM**

What is the difference between object detection and image classification?

Answer

Classification: predict one class for the entire image. Detection: predict class AND bounding box for each object (multiple objects). Two-stage detectors (Faster R-CNN): region proposals then classification. One-stage (YOLO, SSD): directly predict boxes and classes on a grid—faster but historically less accurate. DETR uses transformer-based end-to-end detection without anchor boxes.

Q34.**MEDIUM**

What is self-supervised learning in computer vision?

Answer

SSL trains representations without human labels using the data itself as supervision. Methods: (1) Contrastive: SimCLR, MoCo—contrast augmented views. (2) Masked autoencoding: MAE—mask random patches, reconstruct pixels. (3) Predictive: DINO/DINOv2—predict masked patch features. SSL representations rival supervised pretraining on many benchmarks.

Q35.**MEDIUM**

What is group normalisation?

Answer

GroupNorm divides channels into G groups and normalises within each group per sample. Intermediate between LayerNorm ($G=1$, all channels) and InstanceNorm ($G=C$, each channel). Independent of batch size—useful in object detection/segmentation with small batches. Avoids BatchNorm's batch-size sensitivity without full LayerNorm's channel-mixing.

Q36.**MEDIUM**

Explain the encoder-decoder architecture with cross-attention.

Answer

Encoder: processes source sequence with bidirectional self-attention, outputs context embeddings. Decoder: generates target sequence; each decoder block has (1) masked causal self-attention over previous outputs, (2) cross-attention where queries = decoder, keys/values = encoder output, (3) FFN. Cross-attention allows decoder to focus on relevant encoder positions at each generation step.

Q37.**MEDIUM**

What is EfficientNet and how does compound scaling work?

Answer

EfficientNet (Tan & Le, 2019) scales width (channels), depth (layers), and resolution together using a compound coefficient ϕ : width $\propto \alpha^\phi$, depth $\propto \beta^\phi$, resolution $\propto \gamma^\phi$ with $\alpha\beta^2\gamma^2 \approx 2$ (constant FLOP constraint per doubling ϕ). α, β, γ found by small grid search. Achieves better accuracy/FLOP trade-off than scaling only one dimension.

Q38.**MEDIUM**

What is the role of the feed-forward network (FFN) in a Transformer?

Answer

Each transformer block's FFN applies two linear transformations with a non-linearity: $\text{FFN}(x) = W_1 \cdot \text{GELU}(W_2 x + b_2) + b_1$. Dimension expanded 4× (e.g., $512 \rightarrow 2048 \rightarrow 512$). It operates position-wise independently (no token mixing). Thought to store factual knowledge in the MLP weights (Geva et al.); attention handles token routing, FFN provides memory.

Q39.**MEDIUM**

What is the difference between autoencoder and VAE?

Answer

Autoencoder: encoder maps $x \rightarrow z$ (deterministic), decoder maps $z \rightarrow x$; trained with reconstruction loss. Latent space may not be smooth. VAE (Variational Autoencoder): encoder maps $x \rightarrow \mu(x), \sigma(x)$ (parameters of Gaussian $q(z|x)$); z sampled via reparameterisation $z = \mu + \sigma \cdot \epsilon$. Loss = reconstruction + $\text{KL}(q||N(0,I))$. Forces structured, continuous latent space enabling generation.

Q40.**MEDIUM**

What is neural architecture search (NAS)?

Answer

NAS automates the design of neural network architectures. Search space: layer types, connections, kernel sizes. Search strategies: reinforcement learning (training a controller), evolutionary algorithms, gradient-based (DARTS—differentiable architecture search). Evaluation: proxy metrics (fewer epochs) or weight sharing (one-shot NAS). Found architectures (EfficientNet, NASNet) often outperform handcrafted ones.

Q41.**HARD**

Why do larger models show emergent abilities?

Answer

Emergent abilities appear discontinuously above a capability threshold (Wei et al., 2022). Hypothesis: tasks require a minimal level of capability (e.g., chain of reasoning steps) that only becomes achievable at sufficient scale. Alternative view (Schaeffer et al., 2023): emergent abilities are partially artefacts of discontinuous metrics (pass@1 on exact-match) that smooth out under continuous metrics. Both views have merit; the mechanism is still debated.

Q42.**HARD**

Explain the mathematical formulation of multi-head attention.

Answer

For head i : $Q_i = X \cdot W_Q$, $K_i = X \cdot W_K$, $V_i = X \cdot W_V$. $\text{head}_i = \text{softmax}(Q_i K_i^T / \sqrt{d_k}) V_i$. Output = $\text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$. Each head operates in $d_k = d_{\text{model}}/h$ dimensions. Total compute $\approx$ same as one large attention head but each head specialises in different relation types. W_O projects concatenated heads back to d_{model} .

Q43.**HARD**

How does Rotary Position Embedding (RoPE) work?

Answer

RoPE encodes position by rotating query/key vectors in 2D subspaces: for each pair (q_{2i}, q_{2i+1}) , apply rotation matrix $R(m\theta_i)$ where m is position and $\theta_i = 10000^{(-2i/d)}$. The dot product $Q \cdot K$ becomes a function of relative position $(m-n)$ only—enables length extrapolation. Implemented as complex multiplication for efficiency. Used in LLaMA 1/2/3, Falcon, Mistral.

Q44.**HARD**

Explain the diffusion model framework.

Answer

Diffusion models define a forward process that gradually adds Gaussian noise: $q(x_{t+1}|x_t) = N(\sqrt{1-\beta_t}x_t + \sqrt{\beta_t}\epsilon, \beta_t I)$, destroying signal over T steps. A neural network (usually UNet with attention) learns the reverse: $p_\theta(x_t|x_{t+1}) = N(\mu_\theta(x_{t+1}, t), \Sigma)$. Training: predict the noise ϵ added at step t (ϵ -parameterisation) via MSE loss. DDPM uses 1000 steps; DDIM enables 50-step deterministic sampling. CLIP-guided / classifier-free guidance steers generation.

Q45.**HARD**

What is the 'grokking' phenomenon in neural networks?

Answer

Grokking (Power et al., 2022): models first memorise training data (near-zero train loss) but poor generalisation; then after many more epochs of training, suddenly 'grok'—generalise correctly. Observed on algorithmic tasks (modular arithmetic). Mechanism: the model learns a compact, generalising algorithm but weight decay slowly amplifies it relative to the memorised solution. Implies training should run much longer than until train loss convergence.

Q46.**HARD**

How does classifier-free guidance work in diffusion models?

Answer

Classifier-free guidance (Ho & Salimans, 2022): train a single diffusion model jointly with and without condition (randomly drop condition with probability p during training). At inference: $\epsilon = \epsilon_{\text{uncond}} + w \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$. Higher guidance scale w : outputs more aligned with condition but less diverse. Avoids needing a separate classifier; enables DALL-E 2, Stable Diffusion. $w=1$ gives standard sampling; $w=7.5$ is typical for text-to-image.

Q47.**HARD**

What is the difference between GANs, VAEs, and diffusion models for image generation?

Answer

GANs: implicit density—generator fools discriminator. Sharp, fast inference, but training instability, mode collapse, no density estimate. VAEs: explicit approximate posterior via ELBO; stable training; blurry outputs due to pixel-wise reconstruction. Diffusion: explicit score-matching; stable training, highest quality and diversity, but slow (many denoising steps). Diffusion models dominate current image/audio generation; GANs still used in video/real-time.

Q48.**HARD**

Explain gradient checkpointing and when to use it.

Answer

Gradient checkpointing (activation checkpointing): during forward pass, discard intermediate activations instead of caching them all. During backward pass, recompute activations on-the-fly from the nearest checkpoint. Trades compute (recompute) for memory: reduces activation memory from $O(L)$ to $O(\sqrt{L})$ for a chain of L layers. Use when GPU memory is the bottleneck (large batch size or long sequences). Implemented in PyTorch via `torch.utils.checkpoint`.

Q49.**HARD**

What is the relationship between attention and graph neural networks?

Answer

Self-attention on a sequence can be viewed as a fully-connected GNN: each token is a node, all-to-all edges. The attention weight $a_{ij} = \text{attention}(q_i, k_j)$ is the edge weight. Message aggregation = weighted sum of values. Sparse attention = sparse graph. GNNs extend this to arbitrary graph structures with neighbourhood aggregation. Graph Transformers (Graphformer) combine both, adding structural encodings (degree, shortest-path distances) to attention.

Q50.**HARD**

What is mixture-of-depths (MoD) architecture?

Answer

MoD (Raposo et al., 2024): allows tokens to skip certain transformer layers. A router predicts which tokens need full processing at each layer; less important tokens bypass that layer entirely using a residual connection. Reduces average FLOPs per token while maintaining model capacity. Complementary to MoE (which selects experts width-wise); MoD operates depth-wise. Enables inference-time FLOP budgeting.

Q51.**HARD**

How does AlphaFold 2 use attention to predict protein structure?

Answer

AlphaFold 2 (Jumper et al., 2021): uses two coupled representations—MSA (multiple sequence alignment) and pair (residue-pair distances). Evoformer block: (1) MSA attention rows (within-sequence) and columns (across sequences). (2) Outer product mean updates pair from MSA. (3) Triangle attention on pairs (ensures 3D consistency). Structure module: iterative rigid body updates using invariant point attention (IPA) operating in 3D frames. End-to-end differentiable from sequence to atom coordinates.

Q52.**HARD**

What is the lottery ticket hypothesis?

Answer

Frankle & Carlin (2019): a randomly initialised dense network contains a small sub-network ('winning ticket') that, when trained in isolation with its original weights, achieves comparable accuracy to the full network. Found by iterative magnitude pruning: train → prune low-magnitude weights → reset remaining to init values → repeat. Implies most parameters in a network are redundant from the start; explains why pruning and distillation work. Strong lottery ticket hypothesis: sparse network can match full network at same compute.

Q53.**HARD**

Explain neural tangent kernel (NTK) theory.

Answer

NTK (Jacot et al., 2018): as network width $\rightarrow \infty$, the training dynamics of a neural network are governed by a fixed kernel (the NTK), and gradient descent on the loss converges to a kernel regression solution. In this infinite-width limit, the network does not learn features—it behaves like a kernel machine. Practically: NTK regime holds for small learning rates and overparameterised networks. Feature learning (and thus transfer learning) requires finite width or large learning rates beyond NTK regime.

Q54.**HARD**

What is rotary embedding, ALiBi, and how do they compare for length generalisation?

Answer

RoPE: rotates Q,K vectors by position-dependent angle—models relative position implicitly. Generalises modestly beyond training length; requires fine-tuning on longer sequences for large extrapolation. ALiBi (Press et al., 2022): adds a linear penalty to attention scores proportional to distance: $\text{score}(i,j) \leftarrow m \cdot (i-j)$ where m is head-dependent slope. No positional encodings in embeddings. ALiBi extrapolates better out-of-box (trained at 1k, runs at 2k+ without retraining). RoPE with position interpolation (Meta) or YaRN extends RoPE to longer contexts.

Q55.**HARD**

How do you train a transformer from scratch efficiently? Describe the key engineering decisions.

Answer

Decisions: (1) Mixed precision (BF16/FP16 + FP32 master weights). (2) FlashAttention-2 for memory-efficient attention. (3) Gradient checkpointing for large sequence length. (4) Fused kernels (fused AdamW, RMSNorm). (5) Tensor + data + pipeline parallelism (3D parallelism). (6) Gradient accumulation to simulate large batches. (7) Warmup + cosine LR schedule. (8) Gradient clipping (max norm ~ 1.0). (9) Compile model (torch.compile). (10) Asynchronous data loading (prefetch). (11) Efficient tokenisation (Rust-based BPE).

Q56.**HARD**

What is the information bottleneck principle in deep learning?

Answer

Tishby et al.: each layer $X \rightarrow Z \rightarrow Y$ should compress the input X (minimize $I(X;Z)$) while preserving task-relevant information (maximise $I(Z;Y)$). During training: first fitting phase— $I(X;Z)$ and $I(Z;Y)$ both increase. Then compression phase— $I(X;Z)$ decreases while $I(Z;Y)$ stays high. The compressed representation is more generalisable. Controversial: the compression phase was disputed; it depends on the non-linearity. Nonetheless the IB framework explains why intermediate representations are useful for generalisation.

Q57.**HARD**

What is sharpness-aware minimisation (SAM)?

Answer

SAM (Foret et al., 2021) seeks parameters in flat minima—regions where the loss is uniformly low in a neighbourhood, which correlates with better generalisation. Two-step per update: (1) perturb weights to worst-case in ϵ -ball: $\blacksquare = w + \epsilon \cdot g/\|g\|$. (2) Compute gradient at $\blacksquare$ and update w . Cost: $2\times$ forward/backward passes. Effectively: seek parameters where the loss landscape is flat. Significant accuracy gains on image classification; used in training some LLMs.

Q58.**HARD**

Explain how Stable Diffusion's latent diffusion architecture reduces compute.

Answer

Instead of diffusing in pixel space ($O(H \times W)$ tokens at full resolution), LDM encodes images into a compact latent space via a VQ-VAE/VAE (e.g., $512 \times 512 \rightarrow 64 \times 64$ latent). Diffusion UNet operates in this latent space— $4\times$ smaller spatial dimensions $\rightarrow 16\times$ fewer tokens $\rightarrow$ drastically cheaper attention. After denoising, the latent is decoded back to pixels. CLIP text embeddings condition the UNet via cross-attention at multiple resolutions. Result: high-quality 512×512 generation in seconds on a consumer GPU.

Q59.**HARD**

What is the role of positional embeddings in Vision Transformers and how do they affect performance?

Answer

ViT uses 1D learnable positional embeddings added to patch embeddings. Without them, ViT is permutation-invariant to patch order—performance drops significantly. 2D-aware PE (row+column sinusoidal) improves fine-tuning slightly. RoPE-ViT and 2D RoPE: rotate patch embeddings using 2D positions, enabling length generalisation. At higher resolutions than training, interpolating 1D PE (bilinear in 2D) partially recovers performance; fine-tuning at target resolution needed for best results.

Quick Reference: Topic Coverage

Topic	Section	Key Questions
Pretraining & Scaling Laws	GenAI	Q3, Q14, Q35
Fine-Tuning (SFT, LoRA, QLoRA)	GenAI	Q11, Q12, Q28
RLHF / DPO / CAI	GenAI	Q8, Q31, Q32, Q38
RAG & Retrieval	GenAI	Q12, Q13, Q34, Q44
Agents & Tool Use	GenAI	Q47, Q55, Q60
MCP & Model Gateways	GenAI	Q48, Q49
Speculative Decoding	GenAI	Q35, Q54
Quantization (GPTQ, AWQ)	GenAI	Q36, Q56
LLM Serving & PagedAttention	GenAI	Q51, Q52
Hallucination Detection	GenAI	Q6, Q43
Regularisation (L1/L2, Dropout)	ML	Q8, Q21
Ensemble Methods	ML	Q7, Q24
Optimisation (SGD, Adam, SAM)	ML	Q5, Q40, DL Q58
Calibration & Evaluation	ML	Q3, Q19, Q27
Causal Inference	ML	Q43
Transformers & Attention	DL	Q17, Q18, Q22
Positional Encodings (RoPE, ALiBi)	DL	Q17, Q53
Diffusion Models	DL	Q52, Q53, Q57
Training Stability	DL	Q21, Q14
Vision Transformers (ViT)	DL	Q26, Q60
Self-supervised / Contrastive	DL	Q27